

LARA — Descripción del Producto y Arquitectura

Documento de referencia para Product Owner y Arquitecto Estado del código: rama `main` · Fecha del documento: 2026-07-10 Fuente: auditoría directa del código (`src/` , `public/`), no de documentación previa.

Este documento describe **qué hace LARA hoy, la lógica detrás de cada feature** y las **limitaciones conocidas**, con el objetivo de servir como base para decidir los próximos pasos de producto y arquitectura. Al final hay una sección de deuda técnica y backlog sugerido.

1. Resumen ejecutivo

LARA es un SaaS de analítica y reporting para PMOs (Project Management Offices) que convierte un archivo de tareas (Excel/CSV/Google Sheets) en:

- 1. **Métricas EVM** (Earned Value Management) calculadas de forma determinística.
- 2. **Análisis de IA** de riesgos, económico y de salud del equipo, generado por un pipeline de agentes Claude.
- 3. **Reportes ejecutivos y técnicos** listos para stakeholders, redactados por IA.
- 4. Un **asistente conversacional** que explica métricas, redacta comunicaciones y simula escenarios "what-if".
- 5. Un **tablero de salud del equipo** que combina carga de trabajo y bienestar (feedback 1-on-1 analizado por IA).

La propuesta de valor central: **un PM (incluso novato) sube su plan de proyecto tal como lo tiene y obtiene, en un solo flujo, el diagnóstico que normalmente requeriría un analista PMO senior** — con lenguaje claro, señales tempranas y recomendaciones accionables.

Público objetivo

- **PM / líder de proyecto** — sube datos, consume reportes, chatea con LARA.
- **PMO / dirección** — vista de portafolio consolidada, semáforos de salud.
- **Líder técnico** — reporte técnico de flujo de entrega.

Estado de madurez

Producto funcional en producción (Render). Los 5 hitos del roadmap original tienen implementación al menos parcial; el pipeline de IA y el tablero de equipo están esencialmente completos. Frontend sin framework (HTML/JS plano) — deuda de UX pendiente.

2. Arquitectura general

2.1 Stack tecnológico

Capa	Tecnología
Backend	Node.js + Express 5 + TypeScript (puerto 3001)
Base de datos	PostgreSQL en Render (SSL), consultas <code>raw pg</code> , sin ORM
IA	Anthropic Claude vía <code>@anthropic-ai/sdk</code> — modelo por defecto <code>claude-opus-4-8</code>
Ingesta de archivos	<code>multer</code> + <code>xlsx</code> (Excel/CSV); import de Google Sheets vía CSV export
Validación	Zod (esquemas en <code>src/config/validation.ts</code>)
Auth	JWT en cookie httpOnly + <code>bcryptjs</code>
Seguridad HTTP	<code>helmet</code> , <code>cors</code> , <code>express-rate-limit</code>
Logging	<code>pino</code> / <code>pino-http</code>
Frontend	HTML + JavaScript vanilla servido estáticamente (sin React, sin bundler, sin build step)
Tests	Jest + <code>supertest</code> (37 archivos de test, cobertura ~81%)

Nota arquitectónica clave: el frontend son páginas HTML estáticas (`public/*.html`) que hacen `fetch` a la API e inyectan JSON en el DOM con template literals. No hay SPA, ni router de cliente, ni estado compartido. Esto es la mayor deuda de UX/escala del producto (ver §10).

2.2 Patrón de diseño

El backend sigue un patrón **Repository + Adapter + Agent**:

- **Agentes** (`src/agents/`): unidades de razonamiento IA con contrato común (`BaseAgent`).
- **Servicios** (`src/services/`): lógica de negocio determinística (cálculos, sin IA).
- **Rutas** (`src/routes/`): endpoints Express, validados con Zod.
- **Adaptadores** (`src/services/adapters/`): abstracción de fuentes de datos (Excel).

- **Repositorios** (`src/repositories/`): acceso a datos por entidad.

2.3 Flujo end-to-end (de archivo a reporte)



Decisión importante: el análisis se dispara de forma **síncrona** dentro de `save-mapping` (el usuario espera a que terminen las llamadas a Claude). Simple de razonar, pero acopla la latencia de la subida a la latencia del pipeline de IA (ver §10).

3. Modelo de datos

Tablas principales (creadas/migradas de forma idempotente en `src/db-migrate.ts` al arranque):

Tabla	Propósito	Notas
users	Cuentas	id VARCHAR (user_<timestamp>), role (user/analyst/admin), status (pending_approval/active/rejected)
project_data	Un proyecto por usuario	id SERIAL (usado por la UI) + projectid INT de negocio + user_id
ai_analyses	Historial append-only de análisis	agenttype ∈ { normalization , combined }; output JSONB; user_id
team_members	Miembros del equipo por proyecto	Auto-poblados desde "assignee"; latest_wellbeing_score , latest_sentiment , last_feedback_at
team_feedback_notes	Bitácora de feedback 1-on-1	wellbeing_score , sentiment , ai_reasoning (del wellbeingAgent)
password_resets	Tokens de reseteo	—
organization_config	Branding (colores, logo)	Por organización

Puntos no obvios del modelo

- **ai_analyses** es **historial append-only** (no upsert): cada normalización, cada re-análisis y cada snapshot generan filas nuevas. Las lecturas toman el `MAX(id) / ORDER BY generatedat DESC LIMIT 1`.
- **Dos convenciones de "projectid"** coexisten y son una fuente recurrente de confusión:
- En `data.ts` y `team.ts`, el `:projectId` de la URL es `project_data.id`.
- En `analysis.ts` y en el orquestador, es el `projectId` de negocio.
- **projectId de negocio** = `Math.floor(Date.now()/1000)` → enumerable y colisionable si dos usuarios suben en el mismo segundo. Mitigado escapando **todas** las lecturas por `user_id` (ver §8).
- **Sin ORM:** todo es SQL parametrizado directo. Facilita entender qué corre; dificulta refactors amplios.

4. Features implementados

Organizados por los 5 hitos del roadmap original + features transversales. Cada uno con **qué hace** y **la lógica detrás**.

Hito 1 — Autenticación y acceso · ~60%

Qué hace: login con email/contraseña, registro con aprobación manual del admin, reseteo de contraseña, roles.

Lógica: - JWT en cookie `httpOnly` (`auth_token` , `sameSite: strict` , `secure` en prod) → mitiga XSS y CSRF. - Contraseñas con `bcrypt` (cost 12), mínimo 8 caracteres, emails normalizados a minúsculas. - **Mensaje de error genérico** ("Invalid credentials") → evita enumeración de usuarios. - Registro deja al usuario en `pending_approval` ; un **admin lo aprueba** desde el panel en `projects.html` . Usuarios rechazados no pueden entrar. - Dos middlewares: `requireAuth` (rutas de usuario, redirige a `/login`) y `adminAuthMiddleware` (rutas admin, responde 401/403 JSON). - Rate limiting: auth 20 req/15 min por IP.

Pendiente: "remember me", bloqueo por cuenta tras N intentos fallidos (hoy solo hay rate limit por IP), y flujo SPA sin recarga (el login hace `window.location.href`).

Hito 2 — Ingesta de datos · ~55–70%

Qué hace: sube Excel, CSV o importa Google Sheets; la IA sugiere el mapeo de columnas; el usuario confirma; se normaliza y valida.

Lógica: 1. `normalizationAgent` (Claude) recibe headers + 3 filas de muestra y mapea cada columna a un campo estándar: `project_name` , `status` , `estimated_cost` , `actual_cost` , `progress_percent` , `start_date` , `end_date` , `risks` , `assignee` . Devuelve JSON con confianza y razonamiento por columna. Tiene **retry con backoff exponencial** (1s/2s/4s) y validación Zod de cada sugerencia. 2. `transformDataset` / `transformRow` normaliza tipos con fallbacks robustos: - Costos: soporta `$1,234.50` , formato europeo `1.234,50` , códigos de moneda (`USD`), símbolos. - Progreso: detecta si viene en decimal (0.85) o porcentaje (85) mirando el dataset completo, y lo clampa a 0–100. - Fechas: ISO, DD/MM/YYYY, MM/DD/YYYY y **fechas seriales de Excel**; con defaults si falta. - Filas sin `project_name` se descartan. 3. `calculateDIS` (**Data Integrity Score**): puntúa 0–100 la **calidad del dato** subido — % de cobertura por campo, ponderando doble los campos mapeados. Devuelve grado A–F ("Excelente" a "Crítica"). Es el mecanismo para que el usuario sepa cuánto confiar en el análisis. 4. Todo el pipeline de ingesta se comparte entre Excel, CSV y Google Sheets: la importación de GSheet descarga el CSV público, lo escribe como archivo temporal y **reutiliza exactamente el mismo flujo** de detect-columns → mapeo → save-mapping.

Fortaleza: el mapeo semántico por IA es sólido y tolerante a datos desordenados. **Pendiente:** no hay drag & drop, no hay preview visible de filas (se obtienen pero no se renderizan), el mapeo es por dropdowns (no la UI de tarjetas del diseño original).

Hito 3 — Métricas, Gantt y simulación "What-if" · ~50%

Qué hace: calcula EVM completo, renderiza un Gantt con fechas reales y permite simular escenarios.

Lógica — EVM (`metricsCalculator.ts`), 100% determinístico: A partir de las tareas normalizadas agrega costos planeados/reales, progreso ponderado y timeline (fecha más temprana → más tardía), y calcula:

Métrica	Fórmula
BAC	Σ costo estimado
AC	Σ costo real
PV	(% tiempo transcurrido) × BAC
EV	(% avance real) × BAC
CV	EV – AC
SV	EV – PV
CPI	EV / AC
SPI	EV / PV
EAC	BAC / CPI
VAC	BAC – EAC
TCPI	(BAC – EV) / (BAC – AC)
ROI	((EV – AC) / AC) × 100

Métricas por framework (`frameworkMetrics.ts`): además de EVM, calcula tarjetas específicas según la metodología elegida (Scrum, Kanban, Waterfall, SAFe): - **Scrum:** Velocity (binning de tareas cerradas en sprints de 14 días), Sprint Completion Rate, Team Efficiency. - **Kanban:** Cycle Time, Lead Time, WIP actual, Flow Efficiency, Throughput. - **Waterfall:** Fases completadas, tareas atrasadas, adherencia al plan, ruta crítica (proxy). - **SAFe:** PPM (Program Predictability), Flow Load, equipos estimados, PI Success Rate.

Cada framework genera además **insights** en texto (ej. "WIP alto — riesgo de cuello de botella").

Simulación What-if (`scenarioSimulator.ts` + `/api/chat/simulate`): Modelo híbrido en 3 pasos: 1. Claude **parsea** la pregunta en lenguaje natural → `SimulationDelta` estructurado (tipo + semanas/porcentaje). 2. **La matemática la hace código puro** (sin IA): recalcula EVM para 5 tipos de escenario — retraso de cronograma, aceleración, aumento de presupuesto, reducción de alcance, refuerzo de equipo. 3. Claude **narra** el resultado numérico en lenguaje claro para el PM.

Este patrón "**IA para interpretar y narrar, código para calcular**" es un principio de diseño recurrente y deliberado en LARA — evita que el modelo invente números.

Pendiente: el Gantt no tiene zoom, ni filtro de atrasados, ni colapso de fases. El What-if es por botones ($\pm 5\%$ / $\pm 10\%$), no drag-to-simulate, y **no se persiste** (sin sandbox / undo / "aplicar como baseline").

Hito 4 — Pipeline de 3 agentes de IA · ~95%, esencialmente completo

Qué hace: el corazón analítico de LARA. Tres agentes producen el diagnóstico y los reportes.

Orquestación (`multiAgentOrchestrator.ts`): 1. Calcula métricas determinísticas (EVM, framework, early warnings, alertas de equipo). 2. Corre **Risk Agent y Economic Agent en paralelo** (`Promise.all`). 3. Alimenta ambos al **Reporting Agent**, que hace **2 llamadas a Claude en paralelo** (reporte ejecutivo + técnico) — separadas deliberadamente para que no compitan por el mismo presupuesto de tokens. 4. Persiste todo en una fila `combined` de `ai_analyses`.

Risk Agent (`riskAgent.ts`): - Prompt especializado por framework. Devuelve `overallRiskScore` (LOW/MEDIUM/HIGH/CRITICAL), `delayProbability`, ≥ 3 riesgos (título, descripción, probabilidad, impacto) y recomendaciones priorizadas. - **Defensa en profundidad de parseo:** limpia markdown, garantiza estructura mínima, rellena con riesgos genéricos por framework si el modelo falla, deriva `overallRiskScore` de la probabilidad promedio si falta, y tiene un **fallback completo por framework** si el JSON no parsea. `maxTokens: 4096` para evitar truncamiento. - **Integración con equipo (Hito 5.3):** recibe las alertas de desconexión/burnout del equipo y las considera en sus riesgos y recomendaciones.

Economic Agent (`economicAgent.ts`): - Devuelve `budget_status` (ON_TRACK/AT_RISK/CRITICAL), `budget_health`, `daily_burn_rate`, `monthly_resource_cost`, `worst_case_total_cost`, `cost_of_delay` y recomendaciones. Misma estrategia de fallback robusto.

Reporting Agent (`reportingAgent.ts`): - Genera dos documentos en Markdown con estructura fija: - **Reporte Ejecutivo** (para el stakeholder): estado general, métricas de salud RAG, bloqueos, "¿qué necesitas de mí?", nota humana del PM. - **Reporte Técnico** (para el líder técnico): flujo de entrega (deployment frequency, lead time), tablero de salud técnica, bloqueos, capacidad de la arquitectura, acciones tácticas. - **Los colores RAG se pre-calculan en código** y se pasan al prompt, para que reporte y dashboard **nunca se contradigan** sobre el mismo dato.

Hito 5 — Monitor de moral y salud del equipo · Implementado; Fase 3 pendiente

Qué hace: un tablero por proyecto (y consolidado en `team-morale.html`) que muestra la salud de cada persona del equipo en dos ejes independientes, más feedback 1-on-1 analizado por IA.

Lógica (`teamService.ts` + `teamAlerts.ts` + `wellbeingAgent.ts`):

- **5.1 Auto-poblado:** al subir el archivo, extrae nombres distintos de la columna `assignee` y crea miembros de equipo (con filtro `looksLikePersonName` para no ensuciar el tablero con blobs JSON de columnas mal mapeadas). Uniqueness case-insensitive.
- **5.2 Feedback + bienestar:** el PM registra una nota de una reunión 1-on-1 → el `wellbeingAgent` (Claude) la analiza y devuelve `wellbeingScore` (0–1), `sentiment` y `reasoning`. Se guarda en la bitácora y se denormaliza el último score en el miembro. El **GSS (Group Satisfaction Score)** es el promedio de los scores individuales $\times 100$.
- **5.3 Semáforos.** El desglose separa **dos señales que antes estaban fusionadas:**
- **Carga de trabajo (`workloadLevel`):** relativa al promedio del equipo. Rojo = $\geq 1.5 \times$ el promedio de tareas activas o ≥ 1 tarea crítica-atrasada; amarillo = por encima del promedio o ≥ 1 atrasada; verde = en/bajo el promedio sin atrasos.
- **People Health (`peopleHealthLevel`):** por bienestar + recencia del feedback. Rojo = wellbeing < 0.4 o > 45 días sin feedback; amarillo = $0.4-0.7$ o > 30 días; verde = ≥ 0.7 y feedback reciente; **gris none** = nunca recibió feedback (deliberadamente **no** es alarma).
- **Roll-up combinado (`overallLevel`):** el peor de los dos ejes.
- Solo los miembros **no-verdes** se inyectan como alertas al Risk Agent → cierra el ciclo entre salud del equipo y riesgo del proyecto.
- **Gestión manual de recursos:** se puede agregar/eliminar/editar el rol de miembros que no venían en el archivo.

Pendiente (Fase 3): que un agente lea **varios snapshots** de normalización (no solo el último) para calcular el **promedio histórico de carga por persona** y detectar sobrecarga sostenida en el tiempo. La infraestructura de snapshots (Fase 2) ya existe: un dropdown permite asociar una nueva carga a un proyecto existente, acumulando snapshots append-only bajo el mismo `projectid` con `snapshotLabel` (fecha).

Features transversales

Chat / Asistente LARA (`routes/chat.ts`): - Persona: "LARA Assistant", experto PM con 20 años, explica conceptos con analogías, sin condescendencia. - Inyecta el contexto completo del proyecto (métricas, riesgos, económico, alertas, insights) en el prompt. - **Menú de acción inmediata:** cuando la respuesta describe un problema accionable, el modelo emite un bloque `<actions>` con botones ("Redactar mensaje para el equipo", "Preparar reporte ejecutivo", "Simular escenarios"). - **Drafts ("Escudo"):** genera mensajes adaptados a la audiencia — `team` (Slack/Teams, empático, sin jerga EVM) o `clevel` (correo formal, lenguaje de negocio, cifras monetarias).

Portafolio (`portfolioService.ts`): - Vista consolidada de todos los proyectos del usuario con un **Health Score 0–100** compuesto: - CPI aporta hasta 40 pts, SPI hasta 30 pts, base 30 pts. - Penalización por alertas (crítica $\times 10$, alta $\times 5$) y **por el veredicto de riesgo IA** (CRITICAL -25 , HIGH -15 , MEDIUM -5). -

Decisión de diseño importante: el color de salud del portafolio **integra el veredicto cualitativo del Risk Agent**, no solo CPI/SPI. Antes un proyecto podía verse "verde/saludable" en el portafolio mientras su propia página lo marcaba "Risk: HIGH" — ese desajuste se cerró plegando el riesgo IA en la fórmula y mostrándolo como badge explícito.

Early Warning System (`earlyWarning.ts`): detectores determinísticos que producen alertas accionables: tareas estancadas, fuera de fecha, nunca iniciadas, sobre presupuesto, en ruta crítica, WIP alto, progreso rezagado respecto al tiempo. Cada alerta trae severidad, descripción, **acción recomendada** y tareas afectadas.

Branding por organización: colores y logo configurables por org (`organization_config`).

5. El principio de diseño central: IA + determinismo

Vale la pena aislarlo porque debería guiar decisiones futuras:

La IA interpreta, clasifica y comunica. El código calcula.

- Los **números** (EVM, health score, simulaciones, DIS, semáforos) salen de código puro, testeable y reproducible.
- La **IA** se usa para lo que hace bien: mapear columnas semánticamente, evaluar riesgo cualitativo, analizar sentimiento de feedback, y **narrar** en lenguaje humano.
- Cada agente tiene **fallbacks deterministas** y parseo defensivo → el sistema degrada con gracia si el modelo falla o devuelve JSON inválido.

Esto es una fortaleza para confiabilidad y para costos (los números no requieren re-llamar al modelo). Cualquier feature nueva debería respetar esta frontera.

6. Configuración de IA

- Modelo por defecto: `claude-opus-4-8` (configurable con `AI_MODEL`).
- Temperatura: 0.7 (configurable). Max tokens base: 2000; agentes que generan más (risk/economic/reporting) lo suben a 4096; wellbeing lo baja a 300.
- Cliente Anthropic compartido y centralizado en `src/config/anthropic.ts`.
- **Costo por análisis completo:** ~5 llamadas a Claude (normalización + riesgo + económico + 2 reportes) + las del chat/simulación bajo demanda. Con Opus, esto es un driver de costo relevante a escala (ver §10).

7. Internacionalización (i18n)

- **Fase 1 — HECHA:** todo el **contenido generado por IA** (reportes, riesgo, económico, chat, drafts, simulación) sale en **es o en** según el idioma del navegador (`navigator.language` / `Accept-Language`). Infra en `src/config/language.ts` (`normalizeLang` , `languageDirective` , `ragLabel`). Default `es` .
- **Pendiente Fase 2:** mensajes de API, PDF/HTML del servidor, formato de números/fechas por locale.
- **Pendiente Fase 3:** i18n completo de la UI estática (~2.800 líneas de HTML sin framework i18n).
- **Trampa conocida:** el idioma del reporte queda **fijado al generarlo** — verlo en otro idioma requiere regenerarlo. La moneda (`$`) es un tema aparte del idioma.

8. Seguridad y multi-tenancy

- **Aislamiento por usuario:** `project_data` , `ai_analyses` y `team_members` tienen `user_id` explícito, y **todas** las lecturas se escopan `AND user_id = $x` . Esto cierra una clase de fuga cross-tenant (IDOR) que existía cuando el aislamiento era indirecto y `projectid` podía colisionar entre usuarios.
- Cookies httpOnly + sameSite strict; bcrypt cost 12; sin enumeración de usuarios; rate limiting por tipo de endpoint (auth 20/15min, chat 60/min, endpoints pesados 30/15min).
- `helmet` activo (CSP deshabilitado porque la UI usa scripts/estilos inline y CDNs — deuda).
- Validación Zod en los bordes de la API.
- Rutas admin (`/api/debug` , `/api/dev`) detrás de `adminAuthMiddleware` con `role === 'admin'` .

Residuales conocidos: `db.ts` usa `ssl: { rejectUnauthorized: false }` (requerido por el PG gestionado de Render salvo que se aporte un CA cert). CSP deshabilitado.

9. Testing, calidad y despliegue

- **Suite Jest:** 37 archivos de test, cobertura de sentencias ~81%. Cubre rutas, servicios, agentes, middleware, adaptadores y utilidades. Patrón: `supertest` + `jest.mock('.././db')` para rutas; unit tests con colaboradores mockeados para servicios.
- **Deliberadamente sin test:** objetos de datos estáticos (skills demo), bootstrap de conexión DB, utilidades de cron/dev.
- **Despliegue:** Render en `https://pmo-ai-saas.onrender.com` . Config en `render.yaml` (Node web service, `npm install && npm run build && npm start`). Variables sensibles (`DATABASE_URL` , `JWT_SECRET` , `ANTHROPIC_API_KEY` , `ALLOWED_ORIGINS` , `APP_BASE_URL`) se setean en el dashboard.
- **Auto-deploy: OFF.** Cada commit se despliega **manualmente** desde el dashboard de Render. `autoDeploy` en `render.yaml` no aplica en el modo de conexión actual. → **Nada llega a producción automáticamente al hacer merge a main** .
- Migraciones idempotentes al arranque (`runMigrations`), con backfills para esquemas legados. Si la migración falla, el servidor arranca igual (log de error) para no quedar caído.

10. Deuda técnica y limitaciones conocidas

Ordenadas por impacto para las decisiones de PO/Arquitecto.

Arquitectura / escalabilidad

- 1. **Frontend HTML/JS plano (~2.800+ líneas).** Sin framework, sin componentes, sin i18n de UI, sin estado. Es el mayor freno para iterar UX y escalar features de cara al usuario. **Decisión de fondo pendiente:** ¿migrar a un framework (React/Vue/Svelte) o seguir sumando HTML?
- 2. **Análisis síncrono dentro de `save-mapping`.** El usuario espera ~5 llamadas a Claude en serie/paralelo antes de recibir respuesta. No escala bien ni da buena UX en cargas grandes. **Candidato a job asíncrono / cola + estado "procesando".**
- 3. **Costo de IA con Opus.** Cada análisis dispara varias llamadas a un modelo premium. A escala, revisar: caché, modelo más barato para pasos simples (ej. parseo de delta), o batching.
- 4. **Doble convención de `projectId` (`project_data.id` vs `projectid` de negocio)** es una fuente recurrente de bugs. Considerar unificar o documentar/encapsular fuertemente.
- 5. **`projectid = floor(Date.now()/1000)`** — enumerable y colisionable. Hoy mitigado por escopado `user_id`, pero un UUID sería más limpio.

Salud del equipo (Fase 3 y afinamiento)

- 6. **Identidad de persona depende del nombre exacto.** Typos/acentos crean duplicados; no hay normalización fuerte ni UI de fusión.
- 7. A `team_members` **nunca se le quita a nadie** automáticamente: falta un flag "activo en el último snapshot".
- 8. **Promedio histórico de carga (Fase 3)** aún no implementado — el semáforo de carga usa solo el snapshot más reciente vs. promedio del equipo, no la tendencia temporal.

Producto / UX

- 9. **What-if no se persiste** (sin sandbox, undo, ni "aplicar como baseline").
- 10. **Gantt básico:** sin zoom, filtro de atrasados ni colapso de fases.
- 11. **Ingesta sin drag & drop ni preview de filas;** mapeo por dropdowns.
- 12. **i18n incompleto** (Fases 2 y 3): API, PDFs y UI estática siguen mono-idioma; reporte con idioma fijado al generarse.

Operación

- 13. **Auto-deploy manual** → riesgo de que `main` y producción diverjan. Decisión: ¿activar auto-deploy?
- 14. **CSP deshabilitado** y **SSL `rejectUnauthorized: false`** — endurecer cuando se pueda.

11. Backlog sugerido para definir próximos pasos

Propuesta de priorización para la conversación PO + Arquitecto (no es un compromiso, es un punto de partida):

Prioridad	Iniciativa	Tipo	Por qué
Alta	Decisión de framework frontend (o no)	Arquitectura	Desbloquea toda la iteración de UX futura
Alta	Análisis asíncrono (cola + estado "procesando")	Arquitectura	UX + escalabilidad + resiliencia del pipeline IA
Alta	Estrategia de costos de IA (caché / modelo por paso)	Arquitectura	Sostenibilidad económica a escala
Media	Fase 3 salud de equipo (carga histórica multi-snapshot)	Producto	Diferenciador; infra de snapshots ya existe
Media	Normalización/fusión de identidad de personas	Producto/Datos	Precondición para que el tablero de equipo sea confiable
Media	Persistir What-if (sandbox / aplicar baseline)	Producto	Convierte una demo en herramienta de planeación real
Media	i18n Fase 2 (API, PDF, formato de números/fechas)	Producto	Requisito para clientes no hispanohablantes
Baja	Mejoras de Gantt (zoom, filtros, colapso)	Producto	Pulido
Baja	Hardening (CSP, SSL CA, lockout por cuenta, auto-deploy)	Seguridad/Ops	Madurez operativa

Apéndice A — Mapa de código (dónde vive cada cosa)

Área	Archivos clave
Agentes IA	<code>src/agents/{normalizationAgent,riskAgent,economicAgent,reportingAgent,wellbeingAgent,baseAgent}.ts</code>
Orquestación	<code>src/services/multiAgentOrchestrator.ts</code>
Cálculo determinístico	<code>src/services/{metricsCalculator,frameworkMetrics,earlyWarning,scenarioSimulator,teamAlerts,portfolioService,dataTransformer}</code>
Ingesta	<code>src/routes/dataMapping.ts</code> , <code>src/services/{excelParser,googleSheetsImporter,dataIngestService}.ts</code>
Equipo	<code>src/services/teamService.ts</code> , <code>src/routes/team.ts</code>
Chat / simulación / drafts	<code>src/routes/chat.ts</code>
Auth	<code>src/routes/auth.ts</code> , <code>src/middleware/{requireAuth,adminAuthMiddleware}.ts</code> , <code>src/services/jwtService.ts</code>
Config IA / idioma	<code>src/config/{anthropic,language,validation,messages}.ts</code>
Esquema / migraciones	<code>src/db-migrate.ts</code> , <code>src/db.ts</code>
Frontend	<code>public/{index,portfolio,projects,team-morale,login,signup,reset-password}.html</code>

Apéndice B — Endpoints principales de la API

Método	Ruta	Función
POST	<code>/api/auth/{signup,login,logout}</code> · GET <code>/api/auth/me</code>	Autenticación
POST	<code>/api/data/mapping/detect-columns</code>	Sube archivo, IA sugiere mapeo
POST	<code>/api/data/mapping/detect-columns-gsheet</code>	Importa Google Sheet
POST	<code>/api/data/mapping/save-mapping</code>	Confirma mapeo → normaliza → dispara análisis
GET	<code>/api/data/analysis/:projectId/latest</code>	Análisis combinado + health score
GET	<code>/api/data/analysis/:projectId/tasks</code>	Tareas para el Gantt
GET	<code>/api/portfolio</code>	Vista consolidada de portafolio
POST	<code>/api/chat</code> · <code>/api/chat/draft</code> · <code>/api/chat/simulate</code>	Asistente, drafts, what-if
GET	<code>/api/team</code> · <code>/api/team/:projectId</code>	Tablero de equipo
POST/DELETE/PATCH	<code>/api/team/:projectId/members...</code>	Gestión de miembros y feedback
GET	<code>/api/admin/...</code>	Aprobación de usuarios (admin)

Documento generado a partir de una auditoría directa del código en `main` . Para cualquier afirmación específica, el código es la fuente de verdad — este documento apunta a los archivos concretos para verificación.